

CLAUDE CODE

CHEATSHEET v2.1

The most up-to-date reference for Claude Code 2.1 — commands, shortcuts, flags, env vars, models, plugins, skills, MCP, and automation workflows

Last updated [June 12, 2026](#)

What's New

Getting Started

Models & Effort

Keyboard Shortcuts

Configuration

CLI & Flags

Env Variables

Memory & Files

Checkpointing

Slash Commands

Headless Mode

Agent Skills

Plugins

MCP Servers

Git Worktrees

Subagents



Awesome Claude

PDF & PNG

What's New in Claude Code 2.1

Recent Claude Code releases, newest first. Verified against the official [changelog](#). Last reviewed June 12, 2026.

v2.1.170 **Jun 9, 2026** **NEW**

Claude Fable 5 (claude-fable-5) launches — Mythos-class model, select with /model fable.

v2.1.169 **Jun 8, 2026** **NEW**

--safe-mode flag (disable all customizations for troubleshooting), /cd to move session directory, disableBundledSkills setting, post-session hook.

v2.1.166 **Jun 6, 2026** **NEW**

fallbackModel setting (up to 3 fallbacks when primary is unavailable); glob patterns in deny rules.

v2.1.158 **May 30, 2026**

Auto mode on Bedrock, Vertex & Foundry for Opus 4.7+ (opt-in: CLAUDE_CODE_ENABLE_AUTO_MODE=1).

v2.1.154 **May 28, 2026**

Opus 4.8 launches with high-effort default; dynamic /workflows; fast-mode cost reduction; claude --bg --exec for background shell.

v2.1.152 **May 27, 2026**

/code-review --fix applies findings; /simplify split out as cleanup-only; /reload-skills re-scans skills without restart.

v2.1.149 **May 22, 2026**

/usage per-category breakdown (skills, subagents, plugins, MCP servers).



Awesome Claude

Getting Started

Install via curl (macOS/Linux)

Recommended installation method

```
curl -fsSL https://claude.ai/install.sh | bash
```

Install via Homebrew

Install using Homebrew on macOS

```
brew install --cask claude-code
```

Install via PowerShell (Windows)

Install on Windows using PowerShell

```
irm https://claude.ai/install.ps1 | iex
```

Install via npm

Install globally using npm (requires Node.js 18+)

```
npm install -g @anthropic-ai/claude-code
```

Start Claude Code

Launch Claude Code in your project

```
cd your-project && claude
```

Update to Latest

Keep Claude Code up to date

```
claude update
```



Awesome Claude

 Model Aliases (June 2026)

Select with `/model <alias>`, `claude --model <alias>`, the `ANTHROPIC_MODEL` env var, or the `model` setting. Append `[1m]` for the 1M-token context window (e.g. `opus[1m]`).

fable `claude-fable-5`

Mythos-class flagship (Jun 2026). Hardest, longest tasks.

opus `claude-opus-4-8`

Complex reasoning & architecture. Default on Max/Team/API.

sonnet `claude-sonnet-4-6`

Daily coding. Default on Pro / Team Standard.

haiku `claude-haiku-4-5`

Fast & efficient for simple, scoped tasks.

best → Fable 5 / latest Opus

Auto-picks the strongest available model.

opusplan `Opus → Sonnet`

Opus in plan mode, Sonnet for execution.

 Effort Levels — /effort <level>

☐ **low**

Short, latency-sensitive tasks that aren't intelligence-sensitive.

☒ **medium**

Cost-sensitive work that can trade off some intelligence.

☐ **high**



Awesome Claude

★ **xhigh**

Deeper reasoning at higher token spend (default on Opus 4.7).

★★ **max**

Deepest reasoning; can overthink. Session-only.



ultracode

xhigh reasoning + dynamic workflow orchestration. Session-only.

One-off deep reasoning: include `ultrathink` anywhere in your prompt — no session change

Persist effort: `"effortLevel": "high"` in `settings.json` (only low/medium/high/xhigh persist)

Fast mode: `/fast on` or `~0` — faster Opus output, not a smaller model

Keyboard Shortcuts



Bash mode — run a shell command



Mention a file or folder



Run a slash command



Insert a line break



Interrupt Claude mid-response



Open the rewind / edit-previous menu



Awesome Claude

Ctrl+D Exit the session**Ctrl+L** Clear & redraw the screen**Ctrl+O** Toggle the transcript / thinking view**Ctrl+R** Reverse-search command history**Ctrl+T** Toggle the background task list**Ctrl+B** Background a running bash command**Ctrl+V** Paste an image from the clipboard**Ctrl+G** Open the prompt in your \$EDITOR**Ctrl+X Ctrl+K** Kill all background agents (twice to confirm)**Shift+Tab** Cycle permission modes (default → acceptEdits → plan)**⌘P / Alt+P** Switch model**⌘T / Alt+T** Toggle extended thinking**⌘O / Alt+O** Toggle fast mode

Awesome Claude

Configuration

📁 Settings File Locations (Priority Order)

1. **Enterprise:** /etc/claude-code/managed-settings.json (highest priority)
2. **Project Local:** .claude/settings.local.json (personal, git-ignored)
3. **Project Shared:** .claude/settings.json (team settings)
4. **User Global:** ~/.claude/settings.json (personal defaults)

View all settings

List current configuration

```
claude config list
```

Get setting value

Check specific setting

```
claude config get <key>
```

Set configuration

Change a setting value

```
claude config set <key> <value>
```

Add to list setting

Push to arrays like permissions

```
claude config add <key> <value>
```

Remove from list

Remove from array settings

```
claude config remove <key> <value>
```



Awesome Claude

Run `claude --help` for the full list. Most flags work in both interactive and `--print` mode.

`claude` Start an interactive session

`claude "query"` Start with an initial prompt

`claude -p / --print` Headless: run via SDK and exit

`claude -c / --continue` Continue the most recent conversation here

`claude -r / --resume "<session>"` Resume a session by ID or name (picker if none)

`claude update` Update to the latest version

`claude --model <alias>` Set the model (fable/opus/sonnet/haiku/id)

`claude --effort <level>` Set effort: low/medium/high/xhigh/max

`claude --fallback-model <m>` Auto-fallback model(s) when primary is overloaded

`claude --safe-mode` Disable CLAUDE.md, skills, MCP, hooks for troubleshooting (v2.1.169+)

`claude --permission-mode <m>` default/acceptEdits/plan/auto/dontAsk/bypassPermissions



Awesome Claude

`claude --add-dir <path>` Grant access to extra working directories

`claude --output-format <fmt>` Print mode: text / json / stream-json

`claude --allowedTools <list>` Pre-approve tools (no prompts)

`claude --append-system-prompt <txt>` Append text to the default system prompt

`claude --max-turns <n>` Cap agentic turns (print mode)

`claude --max-budget-usd <n>` Stop after spending this much (print mode)

`claude --bg --exec '<cmd>'` Run a shell command as a background agent (v2.1.154+)

`claude -w / --worktree <name>` Start in an isolated git worktree

`claude --mcp-config <file>` Load MCP servers from JSON file(s)

`claude --settings <file>` Load settings JSON (or inline string)

`claude mcp add ...` Add an MCP server (see MCP section)

`claude doctor` Diagnose installation & configuration



Awesome Claude

Set in your shell or in the `env` key of `settings.json`. Common ones for auth, model selection, thinking budget, and feature toggles.

ANTHROPIC_API_KEY API key for direct authentication

ANTHROPIC_MODEL Default model (overridden by `--model`)

ANTHROPIC_DEFAULT_OPUS_MODEL Pin what the 'opus' alias resolves to

ANTHROPIC_DEFAULT_SONNET_MODEL Pin what the 'sonnet' alias resolves to

ANTHROPIC_BASE_URL Override the API endpoint (proxy/gateway)

CLAUDE_CODE_EFFORT_LEVEL Set effort: low/medium/high/xhigh/max/auto

MAX_THINKING_TOKENS Max extended-thinking tokens (0 = off)

ENABLE_PROMPT_CACHING_1H Opt into the 1-hour prompt-cache TTL

CLAUDE_CODE_SAFE_MODE Enable safe mode (same as `--safe-mode`)

CLAUDE_CODE_ENABLE_AUTO_MODE Enable auto mode on Bedrock/Vertex/Foundry

CLAUDE_CODE_DISABLE_1M_CONTEXT Turn off the 1M-token context window



Awesome Claude

CLAUDE_CODE_USE_BEDROCK Route requests through AWS Bedrock

CLAUDE_CODE_USE_VERTEX Route requests through Google Vertex AI

BASH_DEFAULT_TIMEOUT_MS Default timeout for bash commands (def. 120000)

MCP_TIMEOUT MCP server startup timeout (ms)

DISABLE_TELEMETRY Disable telemetry collection

CLAUDECODE Set to 1 inside Claude Code subprocesses

Memory & Files

CLAUDE.md Locations (loaded every session)

Project (team): ./CLAUDE.md or ~/.claude/CLAUDE.md

Project (personal, gitignored): ./CLAUDE.local.md

User (all projects): ~/.claude/CLAUDE.md

Org policy (managed): /etc/claude-code/CLAUDE.md

/init

Generate a starter CLAUDE.md from your codebase.

/memory

Edit CLAUDE.md files, toggle auto-memory, and view entries.

@import syntax



Awesome Claude

Path-scoped rules

Put rules in `.claude/rules/*.md` with a `paths: frontmatter glob` — they load only when matching files are read.

Auto-memory

Claude's own persistent notes live in `~/.claude/projects/<proj>/memory/` ; `MEMORY.md` loads at startup (first 200 lines / 25KB).

Keep it lean

Aim for under ~200 lines — long `CLAUDE.md` files eat context and reduce adherence.

Checkpointing - Undo & Rewind

Automatic Safety Net

Claude Code automatically tracks all file edits, letting you quickly rewind to previous states if changes go wrong.

Rewind Commands

Esc+Esc Open rewind menu to restore previous states

/rewind Open rewind menu via slash command

✨ Rewind Options

Automatic Tracking

Every user prompt creates a new checkpoint

Conversation Rewind

Restore to previous user message while keeping code



Awesome Claude

Code Rewind

Revert file changes while keeping conversation

Full Rewind

Restore both code and conversation to prior state

⚠ Limitations

Bash changes not tracked: Files modified by bash commands (rm, mv, cp) cannot be undone

External changes not tracked: Manual edits outside Claude Code aren't captured

Not version control: Checkpoints complement Git but don't replace it

💡 *Use checkpoints for quick recovery, Git for permanent history*

Slash Commands - Interactive Session Control

⚡ Interactive Session Commands

Control Claude's behavior during interactive sessions. Type commands directly in the Claude Code chat interface.

📋 Built-in Commands

`/clear` Start a fresh conversation (keeps project memory)

`/compact [focus]` Summarize the conversation to free context

`/resume [session]` Resume a conversation by ID or name

`/branch [name]` Fork the conversation at this point (alias `/fork`)



Awesome Claude

`/context [all]` Visualize context-window usage

`/export [file]` Export the conversation as text

`/copy [N]` Copy the last (or Nth) response to clipboard

`/recap` One-line summary of the session so far

`/btw <question>` Ask a side question without bloating history

`/model [model]` Switch model (picker; ← → adjusts effort)

`/effort [level]` Set reasoning effort: low/medium/high/xhigh/max

`/fast [on|off]` Toggle fast mode (faster Opus output)

`/plan [desc]` Enter plan mode

`/goal [condition]` Keep working until a condition is met

`/code-review [effort] [--fix]` Review the diff for bugs + cleanups; --fix applies them

`/simplify [target]` Cleanup-only review (no bug hunting)



Awesome Claude

/review [PR] Review a pull request locally

/run Launch & drive your app to confirm a change works

/verify Confirm a code change behaves as intended

/diff Interactive diff viewer

/loop [interval] [prompt] Run a prompt on a recurring interval (alias /proactive)

/workflows View & manage background multi-agent workflow runs

/batch <instruction> Decompose a large change across parallel worktrees

/schedule [desc] Create a scheduled cloud agent (cron routine)

/config Open settings (alias /settings)

/permissions View or update tool permissions

/mcp Manage MCP servers & OAuth authentication

/agents Create & manage subagents



Awesome Claude

/memory Edit CLAUDE.md & toggle auto-memory

/init Generate a starter CLAUDE.md

/cd <path> Move the session to a new working directory (v2.1.169+)

/add-dir <path> Add an extra working directory

/skills List available skills

/reload-skills Re-scan skills without restarting

/plugin Manage plugins (install/enable/disable)

/voice Toggle push-to-talk voice dictation

/status Show version, model, account & connectivity

/usage Plan limits + per-category cost (alias /cost)

/doctor Diagnose installation & configuration

/login Sign in / switch Anthropic accounts



Awesome Claude

 Custom Slash Command Templates**Basic Project Command**

Create a simple project-specific command

 **Save as:** `.claude/commands/optimize.md`

```
Analyze this code for performance issues and suggest optimizations:
```

Personal Command

Create a user-wide command available in all projects

 **Save as:** `~/.claude/commands/security-review.md`

```
Review this code for security vulnerabilities:
```

Command with All Arguments

Use `$ARGUMENTS` placeholder for all arguments

 **Save as:** `.claude/commands/fix-issue.md`

```
Fix issue #$ARGUMENTS following our coding standards
```

Command with Positional Arguments

Use `$1`, `$2`, `$3` for individual arguments

 **Save as:** `.claude/commands/review-pr.md`

```
---
argument-hint: [pr-number] [priority] [assignee]
description: Review pull request with priority and assignee
---

Review PR #$1 with priority $2 and assign to $3.
Focus on security, performance, and code style.
```

Git Commit Command

Advanced command with bash execution and file references



Awesome Claude

```
---
allowed-tools: Bash(git add:*), Bash(git status:*), Bash(git commit:*)
argument-hint: [message]
description: Create a git commit
---

## Context

- Current git status: !`git status`
- Current git diff: !`git diff HEAD`
- Current branch: !`git branch --show-current`
- Recent commits: !`git log --oneline -10`

## Your task

Based on the above changes, create a single git commit.
```

Command with Model Override

Specify model and disable automatic invocation

📁 Save as: `.claude/commands/deep-analysis.md`

```
---
description: Perform deep code analysis
model: claude-opus-4-8
disable-model-invocation: true
---

Perform a comprehensive analysis of this codebase focusing on:
- Architecture patterns
- Potential scalability issues
- Security vulnerabilities
- Performance bottlenecks
```

💡 Usage Tips

Project commands: Store in `.claude/commands/` (shared with team)

Personal commands: Store in `~/.claude/commands/` (available across all projects)

Arguments: Use `$ARGUMENTS` placeholder for dynamic values

Bash execution: Use `!`command`` to run commands before processing

File references: Use `@filename` to include file contents

MCP commands: Format: `/mcp__server__prompt_name`

Headless Mode - Programmatic Usage



Awesome Claude

and agent integrations.

Key Headless Commands

`claude -p "query"` Non-interactive mode, print result and exit

`claude -p --output-format json "query"` Output in JSON format with metadata

`claude -p --output-format stream-json "query"` Stream JSON messages as they arrive

`claude -c -p "query"` Continue most recent conversation non-interactively

`claude --resume <id> -p "query"` Resume specific session non-interactively

`echo "query" | claude -p` Process piped input

`claude --verbose -p "query"` Enable verbose logging for debugging

`claude --max-turns 3 -p "query"` Limit agentic turns in non-interactive mode

Real-World Examples

SRE Incident Response

Automated incident investigation

```
claude -p "Analyze these errors" \  
  --append-system-prompt "You are an SRE expert" \  
  --output-format json \  
  --allowedTools "Bash,Read,mcp__datadog"
```



Awesome Claude

```
gh pr diff 123 | claude -p \
  --append-system-prompt "You are a security engineer" \
  --output-format json \
  --allowedTools "Read,Grep" > audit.json
```

Multi-turn Session

Maintain context across multiple commands

```
session_id=$(claude -p "Start review" --output-format json | jq -r '.session_id')
claude --resume "$session_id" -p "Check compliance"
claude --resume "$session_id" -p "Generate summary"
```

Output Formats

text (default): Plain text response

json: Structured data with metadata (cost, duration, session_id)

stream-json: Streaming messages for real-time processing

 Use JSON formats for programmatic parsing and automation

Agent Skills - Modular Capabilities

What are Agent Skills?

Skills are modular capabilities that extend Claude's functionality. Unlike slash commands (user-invoked), Skills are model-invoked—Claude autonomously uses them based on context.

Location: `.claude/skills/` (project) or `~/.claude/skills/` (personal)

Structure: Directory with `SKILL.md` + optional scripts and resources

Discovery: Automatic based on description and context

Skill Setup Commands

```
mkdir -p .claude/skills/skill-name
```

 Create project Skill directory

```
mkdir -p ~/.claude/skills/skill-name
```

 Create personal Skill directory


Awesome Claude

 Skill Templates**Simple Skill**

Basic Skill with single file

 [.claude/skills/commit-helper/SKILL.md](#)

```
---
name: generating-commit-messages
description: Generates clear commit messages from git diffs. Use when writing commit messages
or reviewing staged changes.
---

# Generating Commit Messages

## Instructions

1. Run `git diff --staged` to see changes
2. I'll suggest a commit message with:
  - Summary under 50 characters
  - Detailed description
  - Affected components

## Best practices

- Use present tense
- Explain what and why, not how
```

Skill with Tool Permissions

Skill that limits tool access

 [.claude/skills/code-reviewer/SKILL.md](#)

```
---
name: code-reviewer
description: Review code for best practices and potential issues. Use when reviewing code,
checking PRs, or analyzing code quality.
allowed-tools: Read, Grep, Glob
---

# Code Reviewer

## Review checklist

1. Code organization and structure
2. Error handling
3. Performance considerations
```



Awesome Claude

Skills vs Slash Commands

Slash Commands: User-invoked (you type `/command`), simple prompts, single file

Agent Skills: Model-invoked (automatic), complex capabilities, multiple files + scripts

 Use Skills for comprehensive workflows, Commands for quick prompts

Plugins - Extend Claude Code

What are Plugins?

Plugins extend Claude Code with custom commands, agents, hooks, Skills, and MCP servers. Install from marketplaces or create your own.

Plugin Management Commands

`/plugin` Open plugin management interface

`/plugin marketplace add <url-or-path>` Add a plugin marketplace

`/plugin install <name>@<marketplace>` Install a plugin from marketplace

`/plugin enable <name>@<marketplace>` Enable a disabled plugin

`/plugin disable <name>@<marketplace>` Disable a plugin without uninstalling

`/plugin uninstall <name>@<marketplace>` Completely remove a plugin

Quick Start Examples



Add GitHub Marketplace

Awesome Claude

Install Plugin

Install a specific plugin from marketplace

```
/plugin install formatter@your-org
```

Local Development Marketplace

Add local marketplace for plugin development

```
/plugin marketplace add ./dev-marketplace
```

🌟 What Plugins Can Provide

Custom Commands: Add slash commands for your workflows

Specialized Agents: Deploy expert subagents for specific tasks

Agent Skills: Package capabilities Claude can discover

Automation Hooks: Run scripts on Claude Code events

MCP Servers: Connect to external tools and services

MCP Servers & Extensions

🔌 Model Context Protocol (MCP)

Extend Claude Code with external tools and integrations. MCP servers provide additional capabilities like browser automation, database access, and API integrations.

📋 MCP Management Commands

```
claude mcp add <name> <command> [args...]
```

 Add MCP server with command

```
claude mcp add --transport sse <name> <url>
```

 Add SSE transport MCP server

Awesome Claude

`claude mcp list` List installed MCP servers

`claude mcp remove <name>` Remove MCP server

⚡ Popular MCP Integrations

Airtable

Read/write records, manage bases and tables

```
claude mcp add --transport stdio airtable --env AIRTABLE_API_KEY=YOUR_KEY -- npx -y airtable-mcp-server
```

Asana

Interact with your Asana workspace

```
claude mcp add --transport sse asana https://mcp.asana.com/sse
```

Atlassian

Manage Jira tickets and Confluence docs

```
claude mcp add --transport sse atlassian https://mcp.atlassian.com/v1/sse
```

Box

Access enterprise content and automate workflows

```
claude mcp add --transport http box https://mcp.box.com/
```

Canva

Browse, summarize, autofill and generate Canva designs

```
claude mcp add --transport http canva https://mcp.canva.com/mcp
```



Awesome Claude

```
claude mcp add --transport stdio clickup --env CLICKUP_API_KEY=YOUR_KEY --env  
CLICKUP_TEAM_ID=YOUR_TEAM -- npx -y @hauptsache.net/clickup-mcp
```

Cloudflare

Build apps, analyze traffic, manage security

```
claude mcp add --transport http cloudflare https://mcp.cloudflare.com/mcp
```

Cloudinary

Upload, manage, transform media assets

```
claude mcp add --transport http cloudinary https://mcp.cloudinary.com/mcp
```

Daloopa

High quality fundamental financial data

```
claude mcp add --transport http daloopa https://mcp.daloopa.com/server/mcp
```

Figma

Generate better code with full Figma context

```
claude mcp add --transport http figma https://mcp.figma.com/mcp
```

Fireflies

Extract insights from meeting transcripts

```
claude mcp add --transport http fireflies https://api.fireflies.ai/mcp
```

GitHub

Manage repositories, PRs, and issues

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp/
```



Awesome Claude

```
claude mcp add --transport http hubspot https://mcp.hubspot.com/anthropic
```

Hugging Face

Access Hugging Face Hub and Gradio apps

```
claude mcp add --transport http huggingface https://huggingface.co/mcp
```

Intercom

Access customer conversations and tickets

```
claude mcp add --transport http intercom https://mcp.intercom.com/mcp
```

invideo

Build video creation capabilities

```
claude mcp add --transport sse invideo https://mcp.invideo.io/sse
```

Jam

Debug with AI agents accessing Jam recordings

```
claude mcp add --transport http jam https://mcp.jam.dev/mcp
```

Linear

Issue tracking and project management

```
claude mcp add --transport http linear https://mcp.linear.app/mcp
```

Monday

Manage monday.com boards and items

```
claude mcp add --transport http monday https://mcp.monday.com/mcp
```



Awesome Claude

```
claude mcp add --transport http netlify https://netlify-mcp.netlify.app/mcp
```

Notion

Read docs, update pages, manage tasks

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

PayPal

Payment processing and transaction management

```
claude mcp add --transport http paypal https://mcp.paypal.com/mcp
```

Plaid

Banking data and financial account linking

```
claude mcp add --transport sse plaid https://api.dashboard.plaid.com/mcp/sse
```

Playwright

Advanced browser automation

```
claude mcp add --transport stdio playwright -- npx @playwright/mcp@latest
```

Puppeteer

Browser automation

```
claude mcp add --transport stdio puppeteer -- npx -y @modelcontextprotocol/server-puppeteer
```

Sentry

Monitor errors, debug production issues

```
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
```



Awesome Claude

```
claude mcp add --transport http socket https://mcp.socket.dev/
```

Square

Payments, inventory, orders, and more

```
claude mcp add --transport sse square https://mcp.squareup.com/sse
```

Stripe

Payment processing and subscriptions

```
claude mcp add --transport http stripe https://mcp.stripe.com
```

Stytch

Configure and manage authentication services

```
claude mcp add --transport http stytch http://mcp.stytch.dev/mcp
```

Supabase

Database operations

```
claude mcp add --transport stdio supabase --env SUPABASE_ACCESS_TOKEN=YOUR_TOKEN -- npx -y  
@supabase/mcp-server-supabase@latest
```

Vercel

Manage projects, deployments, and logs

```
claude mcp add --transport http vercel https://mcp.vercel.com/
```

Workato

Access applications, workflows, and data

```
claude mcp add --transport http workato https://mcp.workato.com/mcp
```



Awesome Claude

```
claude mcp add --transport http zapier https://mcp.zapier.com/mcp
```

Git Worktrees - Parallel Development

Why Use Git Worktrees with Claude Code?

No Context Loss: Each Claude session keeps full project understanding

Parallel Work: Handle hotfixes while continuing feature development

Safe Experiments: Test risky changes in complete isolation

Model Comparison: Run different AI models on identical tasks

Zero Switching Cost: No more git stash, checkout, restart cycle

Essential Worktree Commands

```
git worktree add <path> <branch>
```

 Create worktree with existing branch

```
git worktree add <path> -b <branch> <start-point>
```

 Create worktree with new branch

```
git worktree list
```

 List all worktrees and their branches

```
git worktree remove <path>
```

 Remove a worktree directory

```
git worktree prune
```

 Clean up worktree references

```
git worktree move <worktree> <new-path>
```

 Move worktree to new location

Powerful Workflow Examples

Parallel Feature Development



Awesome Claude

```
git worktree add ../myapp-auth -b feature/auth main
git worktree add ../myapp-api -b feature/api main
cd ../myapp-auth && claude
cd ../myapp-api && claude
```

Hotfix While Developing

Handle urgent fixes without losing context

```
git worktree add ../myapp-hotfix -b hotfix/critical main
cd ../myapp-hotfix
claude "fix the critical bug"
# Original work continues in main directory
```

Safe Experiment Testing

Test risky changes in isolation

```
git worktree add ../myapp-experiment -b experiment/refactor main
cd ../myapp-experiment
claude "refactor the entire auth system"
# If it fails, just delete the directory
```

Model Comparison

Compare different AI implementations

```
git worktree add ../ml-sonnet -b experiment/sonnet main
git worktree add ../ml-opus -b experiment/opus main
cd ../ml-sonnet && claude --model sonnet
cd ../ml-opus && claude --model opus
```

Code Review Preparation

Polish code without affecting main work

```
git worktree add ../myapp-review -b review/cleanup feature-branch
cd ../myapp-review
claude "add docs, tests, and clean up code style"
git merge review/cleanup
```



Awesome Claude

Why Use Subagents?

Context Preservation: Each subagent has separate context from main conversation

Specialized Expertise: Fine-tuned prompts for specific domains and tasks

Reusable Workflows: Share subagents across projects and with your team

Tool Control: Grant specific tool access per subagent for security

Automatic Delegation: Claude proactively uses appropriate subagents

Subagent Locations

Project subagents: `.claude/agents/` (highest priority, team shared)

User subagents: `~/.claude/agents/` (personal, all projects)

File format: Markdown with YAML frontmatter

Essential Subagent Commands

`/agents` Open subagents management interface

`claude "Use the [name] subagent to..."` Explicitly invoke a specific subagent

`mkdir -p .claude/agents` Create project subagents directory

`mkdir -p ~/.claude/agents` Create user subagents directory

Ready-to-Use Subagent Templates

Code Reviewer

Expert code review after writing/modifying code

 Save as: `.claude/agents/code-reviewer.md`

```
---
name: code-reviewer
description: Expert code review specialist. Proactively reviews code for quality, security, and
maintainability. Use immediately after writing or modifying code.
tools: Read, Grep, Glob, Bash
model: inherit
---
```



Awesome Claude

1. Run git diff to see recent changes
2. Focus on modified files
3. Begin review immediately

Review checklist:

- Code is simple and readable
- Functions and variables are well-named
- No duplicated code
- Proper error handling
- No exposed secrets or API keys
- Input validation implemented
- Good test coverage
- Performance considerations addressed

Provide feedback organized by priority:

- Critical issues (must fix)
- Warnings (should fix)
- Suggestions (consider improving)

Include specific examples of how to fix issues.

Debugger

Debugging specialist for errors and test failures

📁 Save as: [.claude/agents/debugger.md](#)

```
---
name: debugger
description: Debugging specialist for errors, test failures, and unexpected behavior. Use
proactively when encountering any issues.
tools: Read, Edit, Bash, Grep, Glob
model: sonnet
---
```

You are an expert debugger specializing in root cause analysis.

When invoked:

1. Capture error message and stack trace
2. Identify reproduction steps
3. Isolate the failure location
4. Implement minimal fix
5. Verify solution works

Debugging process:

- Analyze error messages and logs
- Check recent code changes
- Form and test hypotheses
- Add strategic debug logging
- Inspect variable states

For each issue, provide:

- Root cause explanation
- Evidence supporting the diagnosis



Awesome Claude

Focus on fixing the underlying issue, not just symptoms.

Data Scientist

Data analysis expert for SQL and BigQuery

📁 Save as: `.claude/agents/data-scientist.md`

```
---
name: data-scientist
description: Data analysis expert for SQL queries, BigQuery operations, and data insights. Use
proactively for data analysis tasks.
tools: Bash, Read, Write
model: opus
---
```

You are a data scientist specializing in SQL and BigQuery analysis.

When invoked:

1. Understand the data analysis requirement
2. Write efficient SQL queries
3. Use BigQuery command line tools (bq) when appropriate
4. Analyze and summarize results
5. Present findings clearly

Key practices:

- Write optimized SQL queries with proper filters
- Use appropriate aggregations and joins
- Include comments explaining complex logic
- Format results for readability
- Provide data-driven recommendations

Always ensure queries are efficient and cost-effective.

💡 Usage Tips

Automatic invocation: Include "proactively" or "MUST BE USED" in descriptions

Explicit invocation: "Use the code-reviewer subagent to check my code"

Chaining: "Use debugger then optimizer subagents"

Tool limitation: Only grant necessary tools for security and focus

Team sharing: Check project subagents into version control

📄 Subagent File Format

```
---
name: your-subagent-name
```



Awesome Claude

Your subagent's system prompt goes here.
Include specific instructions, examples, and constraints.

Define the subagent's role, capabilities, and approach
to solving problems within its domain.

Permissions & Security

Ready-to-Use Permission Templates

Copy complete settings.json configurations for different security levels. Save to `.claude/settings.json` in your project.

MCP Tool Permissions

Wildcards NOT supported: `mcp__github__*` will NOT work

- ✓ Approve all tools from server: `"allow": ["mcp__github"]`
- ✓ Approve specific tool: `"allow": ["mcp__github__get_issue"]`

Enterprise Configuration

Managed settings: `/etc/claude-code/managed-settings.json`

Managed MCP: `/etc/claude-code/managed-mcp.json`

MCP allowlist/denylist: Control which MCP servers employees can use via `allowedMcpServers` and `deniedMcpServers`

 *Denylist takes absolute precedence over allowlist*

Basic Security

Block sensitive files, allow common dev commands

 **Good starting point for most projects**

```
{
  "permissions": {
    "allow": [
      "Bash(npm run *)",
      "Bash(git status)",
      "Bash(git diff)",
      "Bash(git add *)",
      "Bash(git commit *)"
    ]
  },
}
```



Awesome Claude

```
"deny": [
  "Read(/.env*)",
  "Read(/secrets/**)",
  "Read(/**/credentials*)",
  "Bash(rm -rf:*)",
  "Bash(curl:*)"
]
}
```

Strict Security

Maximum security for sensitive projects

💡 [For production systems or sensitive codebases](#)

```
{
  "permissions": {
    "ask": [
      "Bash(*)",
      "Write(*)",
      "Edit(*)"
    ],
    "deny": [
      "Read(/.env*)",
      "Read(/secrets/**)",
      "Read(/config/**)",
      "Read(/**/credentials*)",
      "Read(/**/*.key)",
      "Read(/**/*.pem)",
      "WebFetch",
      "Bash(curl:*)",
      "Bash(wget:*)",
      "Bash(ssh:*)"
    ]
  }
}
```

Slash Command Permissions

Control which slash commands Claude can invoke

💡 [Restrict automated command execution](#)

```
{
  "permissions": {
    "allow": [
      "SlashCommand:/commit",
      "SlashCommand:/review-pr:*"
    ],
    "deny": [
```



Awesome Claude

Enterprise MCP Restrictions

Example enterprise MCP server allowlist/denylist

💡 **Control which MCP servers can be used**

```
{
  "allowedMcpServers": [
    {
      "serverName": "github"
    },
    {
      "serverName": "sentry"
    },
    {
      "serverName": "company-internal"
    }
  ],
  "deniedMcpServers": [
    {
      "serverName": "filesystem"
    }
  ]
}
```

Hooks - Automation & Control

⚠️ Security Warning

Hooks execute shell commands automatically with your credentials. Always review hook code before adding. Malicious hooks can damage your system or exfiltrate data.

Use hooks to automate formatting, logging, notifications, and custom permissions instead of relying on Claude to remember these tasks.

📋 Hook Events

PreToolUse Runs before tool calls (can block them)

PostToolUse Runs after tool calls complete



Awesome Claude

Notification Runs when Claude Code sends notifications

Stop Runs when Claude Code finishes responding

SubagentStop Runs when subagent tasks complete

PreCompact Runs before compact operation

SessionStart Runs when Claude Code starts/resumes session

SessionEnd Runs when Claude Code session ends

🔧 Hook Management Commands

/hooks Open hooks configuration interface

claude --debug Run with debug output to see hook execution

mkdir -p .claude/hooks Create project hooks directory

chmod +x .claude/hooks/script.sh Make hook script executable

🎯 Ready-to-Use Hook Examples

Auto Format Code

Run prettier on edited files



Awesome Claude

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write $FILE_PATH"
          }
        ]
      }
    ]
  }
}
```

Log Activity

Keep a log of all Claude sessions

Event: Stop

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "echo \"$(date): $CONVERSATION_SUMMARY\" >> ~/.claude/activity.log"
          }
        ]
      }
    ]
  }
}
```

Hook Structure & Tips

Settings location: ~/.claude/settings.json (user) or .claude/settings.json (project)

Matchers: Match tool names (e.g., Bash , Edit|Write , *)

Project scripts: Use \$CLAUDE_PROJECT_DIR for relative paths

Exit codes: 0 =success, 2 =block & show to Claude, other=error

JSON input: Hooks receive session data via stdin (use jq to parse)

Debugging: Use claude --debug to see hook execution details



Awesome Claude

KEYBOARD SHORTCUTS

⌘

bash mode prefix

⌘

Mention files/folders

⌘

Line break (backslash + Enter)

ESC

Interrupt Claude

⌘⇧C

Pull output/context

⌘⇧V

Paste JSON

⌘⇧H

History navigation

⌘⇧A

Auto-accept ("yolo mode")

⌘⇧P

Plan mode

⌘⇧L

Quick launch in IDEs

⌘⇧R

Insert file references

ESSENTIAL COMMANDS

npm install -g anthropic-ai/claude-code

Install Claude Code globally

claude update

Update to latest version

claude config set model claude-opus-4.1-20250805

Set Opus 4.1 as default model

claude config set -g theme dark

Enable dark theme globally

SLASH COMMANDS

/clear

Clear history

/init

Initialize project

/model

Change AI model

/config

View/modify config

/cost

Token usage stats

/doctor

Health check

/agents

Manage subagents

/mcp

MCP servers

/memory

Edit CLAUDE.md

/permissions

Security

/review

Code review

/win

Win mode

KEY FILE LOCATIONS

~/.claude/settings.json

User global settings

~/.claude/settings.json

Project settings

~/.claude/commands/

Custom slash commands

~/.claude/agents/

Project subagents

CONFIGURATION

claude config list

View all settings

claude config set <key> <value>

Change setting value

claude config add <key> <value>

Add to array settings

GIT WORKTREES

git workspace add ../app-feature -b feature main

Create new workspace with branch

git workspace list

List all worktrees

git workspace remove <path>

Remove workspace

POPULAR MCP SERVERS

claude mcp add playwright npm playwright/mcp@latest

Browser automation

claude mcp add --transport http context7

https://mcp.context7.com/mcp

Documentation access

HOOKS & AUTOMATION

Hook Events:

PreToolUse

PostToolUse

UserFeedbackStart

Stop

SessionStart

SessionEnd

Settings Location:

~/.claude/settings.json or ~/.claude/settings.json

AI MODELS

Opus 4.1

Claude-opus-4.1-20250805

Most capable

Sonnet 4

Claude-sonnet-4-20250814

Balanced performance

Haike 3.5

Claude-3.5-haiku-20241022

Fastest response

Get the full interactive cheatsheet at [awesomeclaude.ai/code-cheatsheet](#)

Claude Code is developed by Anthropic • This cheatsheet is maintained by awesomeclaude.ai

Printable Version

Special version for printing or keeping as a desktop reference. Contains all essential keyboard shortcuts, commands, configurations, and file locations in a compact format.

Download PNG

Download PDF

View Full Size



Awesome Claude

A W3C browser API letting web apps expose JavaScript functions as AI-callable tools - agents interact via structured typed APIs instead of brittle UI actuation (screenshots, DOM scraping, simulated clicks).

```
navigator.modelContext.registerTool({
  name: "searchProducts",
  description: "Search the catalog",
  execute: async (input) => {
    await search(input.query);
    annotations: { readOnlyHint: true }
  });
```

GOALS

- Human-in-the-loop
- Reuse frontend code
- Replace UI actuation
- Improve accessibility

NON-GOALS

- Headless browsing
- Autonomous agents
- Replace backend MCP
- Site discoverability

```
// Navigator extension
partial interface Navigator {
  [SecureContext, SameObject]
  readonly attribute
    ModelContext modelContext;
};

// ModelContext
[Exposed=Window, SecureContext]
interface ModelContext {
  undefined
    registerTool(ModelContextTool t);
  undefined
    unregisterTool(DOMString name);
};
```

SecureContext
HTTPS only

SameObject
Singleton per page

```
// Tool dictionary
dictionary ModelContextTool {
  required DOMString name;
  required DOMString description;
  object inputSchema;
  required ToolExecuteCallback execute;
  ToolAnnotations annotations;
};

// Execute callback
dictionary ToolAnnotations {
  boolean readOnlyHint = false;
};

// Execute callback
callback ToolExecuteCallback =
  Promise<object> (
    object input,
    ModelContextClient client);

// User interaction
[Exposed=Window, SecureContext]
interface ModelContextClient {
  Promise<any>
    requestUserInteraction(
      UserInteractionCallback cb);
};
```

MODELCONTEXTTOOL FIELDS

Field	Type	Req?	Notes
name	DOMString	YES	Unique key. Duplicate name → InvalidStateError.
description	DOMString	YES	Natural language. Tells agent when and how to use it.
inputSchema	object	NO	JSON Schema for params. Invalid schema throws.
execute	Callback	YES	async (input, client) → object. Runs on tool call.
annotations	object	NO	readOnlyHint: true = no confirmation needed.

inputSchema Example (JSON Schema)

```
{
  "type": "object",
  "properties": {
    "query": { "type": "string", "description": "Search term" },
    "size": { "type": "string", "enum": ["S", "M", "L", "XL"] },
    "limit": { "type": "integer", "minimum": 1 }
  },
  "required": ["query"]
}
```

CODE PATTERNS

1 Read-Only Tool

```
navigator.modelContext
  .registerTool({
    name: "getProducts",
    description: "List all products",
    execute: async (input) => {
      items: await api.list(input)
    },
    annotations: {
      readOnlyHint: true
    }
  });
```

2 State-Modifying Tool

```
navigator.modelContext
  .registerTool({
    name: "addToCart",
    description: "Add item to cart",
    inputSchema: {
      type: "object",
      properties: {
        id: { type: "string" }
      }
    },
    execute: async (input) => {
      cart.add(input.id)
    }
  });
```

3 User Confirmation

```
execute: async (input, client) => {
  const ok = await client
    .requestUserInteraction(
      async () => {
        showConfirmDialog(
          `Delete ${input.id}?`);
      });
  if (!ok) return { cancelled: true };
  return await deleteItem(input.id);
}
```

4 Dynamic Registration

```
function onLogin(user) {
  navigator.modelContext
    .registerTool({
      name: "getOrders",
      execute: () => {
        api.orders(user.id)
      }
    });
}

function onLogout() {
  navigator.modelContext
    .unregisterTool("getOrders");
}
```

OFFICIAL USE CASES (W3C)

Creative Applications

Graphic design platform - agent helps find templates and edit designs while user reviews and approves every step.

`filterTemplates(desc)` `editDesign(instructions)`

E-Commerce / Shopping

Clothing store - agent gets structured JSON product list and updates UI. No screenshots - far more reliable.

`getDresses(size)` `showDresses(ids)`

Developer Tooling (Code Review)

Code review platform - agent fetches CI results and applies suggestions while developer reviews and approves.

`getTryRunStatuses()` `addSuggestedEdit(file, patch)`

WEBMCP VS. OTHER APPROACHES

Shared UI

Reliability

Token cost

Tab required

Best for

WebMCP

Browser API

Yes

High

Low (JSON)

Must be open

Human-in-loop

Backend MCP

Server-side

No

High

Low (JSON)

No - always on

Autonomous

UI Actuation

DOM / screenshots

Yes

Fragile

~2k/screenshot

Must be open

Last resort

USE WEBMCP WHEN

- User is in the browser tab
- Reusing existing frontend code
- Human oversight is required

USE BACKEND MCP WHEN

- No browser tab needed
- Always-available tools
- Fully autonomous pipeline

USE A2A WHEN

- Agent ↔ Agent comms
- Long-running tasks
- No human in the loop

SECURITY & TRUST MODEL

1 Site → Browser

Site registers tools, exposes capabilities. Browser prompts user for permission.

2 Agent → Site

Agent sends untrusted input; output may be sensitive. Browser mediates all calls.

Model Poisoning

Crafted tools could expose hidden data or manipulate the agent.

iFrames Blocked

Only top-level tabs can register tools. Embedded iFrames excluded.

Cross-Origin

Browser must disclose which apps are invoked when data crosses origins.

DEVELOPER SECURITY CHECKLIST

Treat agent inputs as untrusted

Unregister tools on logout

Return only user-visible data

Set readOnlyHint on safe tools

FUTURE ROADMAP & OPEN QUESTIONS

Declarative WebMCP

Register tools via HTML form elements. Schema synthesized automatically from form fields - no JS needed.

Background Providers

Tools that run without a visible UI - show only a notification. Combined with web app launch handler API.

Open Design Questions

Main thread perf - sequential tool calls may block the UI. Under investigation.

Permission UX - exact prompt design and "always allow" granularity need coordination.

iFrame delegation - how parent pages grant capabilities to embedded iFrames.

DEVELOPER RESOURCES

WebMCP Inspector

Inspect & test tools registered on any page
webmcpinspector.com

W3C Specification

Official draft - Web Machine Learning CG
webmachinelearning.github.io/webmcp



WebMCP Cheat Sheet W3C

Complete quick reference to the `navigator.modelContext` API — register JavaScript functions as AI-callable browser tools.

- Core API
- Code Patterns
- ModelContext Fields
- Security Model
- Use Cases

View WebMCP Cheat Sheet



AWESOME CLAUDE

Curated collection of the best Claude AI tools, integrations, and resources for developers.

Quick Links

- Awesome List
- Code Cheatsheet
- Awesome Claude Code
- AnnotateWeb
- IframeTest

Contribute

- Report Issues
- Submit PR
- Contributing Guide



Awesome Claude

© 2026 Awesome Claude MIT License

Maintained by **webfuse** 

